VoiceLedger integration - proposal

Status: proposal for VoiceLedger to review. `farmer.linked` is implemented and running against Stellar Testnet. `farmer.consent`, `farmer.profile` and `delivery.created` are defined here so both sides can build in parallel; our side implements them in week 2 (Nov 23-27) and returns `422 UNSUPPORTED_EVENT` until then. There is no shared environment yet: until we share a staging URL, build against this document and check your signing code with the test vectors.

Pilot scope

This is a closed pilot with participants selected from cooperative-provided lists. CofiBlocks will preload their existing data before onboarding. We need VoiceLedger to provide a simple Telegram experience for these farmers: link their Telegram account once, collect consent and the limited additional farm information described below, and register coffee deliveries through guided voice or button-based interactions. Farmers should not need passwords, wallets, or technical knowledge.

Telegram is the farmers' only point of contact with the system. CofiBlocks manages the records, the traceability of each batch and every blockchain operation; the bot never deals with wallets or transactions.

What we ask VoiceLedger to build

1. **Assisted onboarding, once per farmer.** The farmer is validated against their cooperative's list with the cooperative's help, and the bot then sends `farmer.linked` (see Linking a farmer). After that the bot keeps the link between the farmer's `telegram_user_id` and the `farmer_ref` we return: the farmer never types their ID again.
2. **Consent.** Show the consent texts we provide, record the farmer's explicit answer and when it was given, and send `farmer.consent`.
3. **A short farm questionnaire and corrections.** Follow the fixed questionnaire CofiBlocks provides based on the existing lists (see Existing participant data). Do not repeat the registration form. Send additional farm information or corrections through `farmer.profile`.
4. **Deliveries.** Collect the collection center and the estimated quantity, confirm them with the farmer, and send `delivery.created`.
5. **Weight confirmation.** Tell the farmer the weight the cooperative recorded for their delivery (see Notifications).
6. **The conversation rules below**, and the bot's texts in the farmers' languages.
7. **The technical contract:** the four events in this format, signed, with the retry rules and error codes in this document.

Conversation rules

- Remember the language the farmer chose.
- Accept voice, and offer buttons to confirm or correct what was understood.
- Ask one question at a time.
- Before sending a delivery, read back what was understood (center and quantity) and wait for the farmer's confirmation.
- For a network failure or a 5xx, explain that confirmation is pending and retry in the background with the same `event_id`. For `202 processing`, follow the status-check rules below. For a final business error, help correct the data or contact the cooperative; do not retry unchanged invalid data indefinitely. Never ask the farmer to repeat an action just because confirmation is delayed.
- Always offer a way out to get help from the cooperative.
- The main confirmation says what was recorded, in plain words ("Your delivery of about 45 kg at Kirkir was recorded"). The blockchain explorer link is secondary and optional.

What CofiBlocks still owes VoiceLedger

Input	Status
Consent texts in Afaan Oromoo, Amharic and English, with their version IDs	To be sent before week 2
Import of the selected participants' existing data	CofiBlocks must complete this before onboarding; the workbooks contain more fields than our current importer loads
Short farm questionnaire and the limited existing values needed by the bot	Based on the lists below; CofiBlocks will provide these before profile testing, without requiring a general-purpose profile lookup API
Meaning and units of the existing land areas, and meaning of the coordinates	Confirm the actual area units and local conversion factor with the cooperatives, and whether these describe the coffee farm or the beneficiary's residence/total land
Final practice questions and answer choices	Still to confirm; the `practices` example below is provisional
Staging URL and the per-environment secret	When staging is up
Notification mechanism for the weight confirmation (webhook or polling)	To agree with you
Confirmed spelling of the collection center names	Being confirmed with the cooperatives

Who does what

- **VoiceLedger** runs the farmers' Telegram bot: the conversation, voice recognition in the local languages (Afaan Oromoo, Amharic), the bot's texts and translations. It turns what the farmer says or chooses into the JSON events below and sends them to us.
- **CofiBlocks** receives the events, checks them against the cooperatives' lists, stores the full record privately and writes the on-chain record on Stellar. Farmers have no wallet: a CofiBlocks backend account (the *farmer agent*) signs every farmer action. On chain, a farmer is only their pseudonymous `farmer_ref` (e.g. `KK-ID001`); names, phone numbers and GPS points never go on chain.
- **The cooperatives** provide the farmer lists, help with onboarding and resolve problems the bot cannot (a wrong ID, an account linked to the wrong person).

Existing participant data

We have reviewed the **Dandi Misoma Cooperative Farmers list and Profile** and **Kilole Kirkir Cooperative farmers list and Profile** workbooks linked in our project workspace. They already include the following. CofiBlocks handles the import; VoiceLedger should not ask farmers to fill out these lists again.

Data in the lists	What the bot needs to do
Cooperative and farmer registration ID; farmer, father and grandfather names	Use the assisted onboarding to link the existing participant; do not create another registration
Region, zone, woreda, kebele and village	Do not ask again as part of the initial questionnaire
Household size and other family/demographic data	Do not collect again; these are not new pilot questions
Phone number, where available	Optional; missing phone numbers must not block onboarding

Data in the lists	What the bot needs to do
Owned and rented land area	Do not ask for the same area again by default; CofiBlocks must confirm the actual units before interpreting or converting the imported values
Beneficiary latitude, longitude and elevation	Coordinates are present in both lists; do not request a new location by default

The coordinates are labelled **beneficiary location**, and the areas describe **owned/rented land**, without identifying individual coffee plots. We must confirm their meaning with the cooperatives before treating them as coffee-plot coordinates or coffee-growing area. A zero land value must not automatically be treated as missing: it may mean the farmer has no land of that tenure type.

The source area headers say **"Owned unit land in Hectar?"** and **"Rent unit land in Hectar?"**. These labels alone do not confirm the actual units of the recorded values or the units farmers use. Preserve the source values until the cooperative confirms their units. In the conversation, accept the local unit used by the farmer; do not require the farmer to convert to hectares.

What VoiceLedger does need to collect is the Telegram account link, the preferred language, explicit consent, the agreed practice answers, and each new delivery. Ask for coffee-plot area or location only where CofiBlocks' final questionnaire identifies a genuine gap or the farmer needs to correct it. The lists do not contain practice answers; the intervention/training columns are not farming practices.

For this pilot, CofiBlocks will supply a short fixed questionnaire and any participant-specific exceptions or existing values needed for confirmation before joint testing. VoiceLedger does not need to inspect the full source workbooks or build a dynamic questionnaire engine.

Linking a farmer

Before onboarding, CofiBlocks will preload the selected farmers from the cooperatives' lists, including the existing information described above. Linking is not self-registration: a farmer cannot join by typing any ID.

The proposed flow:

1. A cooperative member (collector or field staff) helps the farmer start the bot and choose their language.
2. The farmer is validated against the cooperative's list: their cooperative ID, and their name or phone where the list has them. The phone is optional, because some list entries do not have one.
3. The bot sends `farmer.linked` with the cooperative code, the farmer's ID and the Telegram account.
4. The bot stores the returned `farmer_ref` for that Telegram account and uses it in every later event.

To agree with you: how the cooperative's validation in step 2 is represented in the bot and in the event. For example, the cooperative member confirms the farmer from their own Telegram account, or the cooperative hands each farmer a one-time onboarding code. We would like your proposal; once agreed, we add the field to `farmer.linked`. The current fields keep working in the meantime.

Transport

- **Endpoint:** `POST {API}/api/integrations/voiceledger/events`, one event per request, `Content-Type: application/json`.
- **Status of an event:** `GET {API}/api/integrations/voiceledger/events/{event_id}`.
- **Environments:** a staging URL and a per-environment secret will be shared privately once staging is up. Production gets its own secret.

Authentication

Every request (including `GET`) carries two headers:

Header	Value
`X-VoiceLedger-Timestamp`	Unix time in seconds when the request was signed.
`X-VoiceLedger-Signature`	`sha256=` + hex of `HMAC-SHA256(secret, "<timestamp>.<METHOD>.<path>.<raw body>")`.

- `METHOD` is upper case (`POST`, `GET`). `path` is the request path exactly as sent, with the query string if any (e.g. `/api/integrations/voiceledger/events`). For `GET`, the body is empty.
- Signing the method and the path means a captured request cannot be pointed at another route or another event.
- Requests more than **5 minutes** away from our clock are rejected (`STALE_TIMESTAMP`), so a captured request cannot be replayed later.
- Sign the exact bytes you send. Example in Node.js:

```
const path = '/api/integrations/voiceledger/events';
const body = JSON.stringify(event);
const timestamp = String(Math.floor(Date.now() / 1000));
const signature = 'sha256=' + crypto.createHmac('sha256', secret)
  .update(`${timestamp}.POST.${path}.`).update(body).digest('hex');
```

- Authentication errors use HTTP `401` with `code` `MISSING_SIGNATURE`, `STALE_TIMESTAMP` or `INVALID_SIGNATURE`; `503` `INTEGRATION_DISABLED` means the secret is not configured on our side.

Test vectors

Use them to check your signing code before there is an environment to call. They come from our backend's own signing function.

	`POST` **an event**	`GET` **its status**
Secret	`test-secret-do-not-use`	`test-secret-do-not-use`
Timestamp	`1795000000`	`1795000000`
Path	`/api/integrations/voiceledger/events`	`/api/integrations/voiceledger/events/9b2f3c1e-7d1a-4c55-9a51-2f0f4f6f8a10`
Body	see below	empty
Signature	`sha256=a376c637bedb0430dbe6db7299b89302eb0c141de132551bf7b6df5a4d3c1c0c`	`sha256=2f38a4935ee780b76474ede72df3c4fc1fa983daa38e35f0a016abc1785a5431`

Body of the `POST`, exactly these bytes (one line, no trailing newline):

```
{"event_id":"9b2f3c1e-7d1a-4c55-9a51-2f0f4f6f8a10","type":"farmer.linked","data":{"coop_code":"KK","farmer_id":"ID001","telegram_user_id":"123456789"}}
```

The signature proves the request comes from VoiceLedger. It does not prove that the person behind a Telegram account is the farmer on the list; that is what the assisted onboarding is for.

Retries and idempotency

Every event has an `event_id` chosen by VoiceLedger (a UUID is fine).

- **Retry with the same** `event_id` (timeouts, 5xx, network errors): we return the stored answer and do not write on chain again. A retry may re-serialize the JSON or refresh `occurred_at`; only `type` and `data` are compared.
- **Same** `event_id` **with a different** `type` **or** `data`**:** rejected with `409 EVENT_ID_REUSED`.
- **A new farmer action** (for example a second delivery, or a correction to the profile) needs a **new** `event_id`.
- Business errors (4xx) are final for that `event_id`. Server errors (5xx) are retried when the same `event_id` arrives again.
- `503 REGISTRATION_PENDING` means the on-chain transaction was sent but is not confirmed yet: retry the same `event_id` a minute later. We never send a second transaction for it until the network proves the first one never landed.
- `503 STELLAR_RPC_UNAVAILABLE` and `503 TRANSACTION_NOT_FOUND` mean the Stellar network could not be read at that moment: retry the same `event_id` later.
- If two copies arrive at the same time, one is processed and the other gets `202` with `"status": "processing"`; check `GET .../events/{event_id}` a few seconds later. If it is still processing after more than two minutes, resend the original `POST` with the same `event_id`, `type` and `data` and fresh signing headers. `GET` only reads the stored status; it does not restart interrupted processing.

Envelope

```
{
  "event_id": "9b2f3c1e-7d1a-4c55-9a51-2f0f4f6f8a10",
  "type": "farmer.linked",
  "occurred_at": "2026-11-16T09:30:00Z",
  "data": { }
}
```

Field	Type	Notes
`event_id`	string, 1-128	VoiceLedger's ID for the event.
`type`	string	One of the event types below.
`occurred_at`	ISO 8601, optional	When it happened on VoiceLedger's side. Informational.
`data`	object	Depends on `type`.

Responses

Success (`200`):

```
{
  "event_id": "9b2f3c1e-7d1a-4c55-9a51-2f0f4f6f8a10",
  "type": "farmer.linked",
  "status": "succeeded",
  "result": { }
}
```

Failure (`4xx`/`5xx`):

```
{
  "event_id": "9b2f3c1e-7d1a-4c55-9a51-2f0f4f6f8a10",
  "type": "farmer.linked",
  "status": "failed",
  "error": { "code": "FARMER_NOT_FOUND", "message": "Farmer ID ID999 is not on KK's list" }
}
```

`error.code` is stable; use it to decide what the bot tells the farmer. `error.message` is for logs, not for farmers. For a 5xx or a `processing` answer, tell the farmer it is being recorded and retry in the background. `INVALID_PAYLOAD` includes a `details` array with the fields that failed validation.

Reference data

Cooperatives and collection centers (Gomma woreda, Jimma zone). `coop_code` and `collection_center` take these values; names are case-insensitive.

coop_code	Cooperative	Collection centers
KK	Kilole Kirkir	Kirkir, Simbo, Madha Bereha, Nagade
DM	Dandi Misoma	Sayiti, Kiltu Holka, Amuma
KN	Kananisa	Kochi, Kiltu Bada, Buna Melka, Yeddu Yeriyo

The spelling of some center names is still being confirmed with the cooperatives; we will tell you if any changes.

Farmer IDs are assigned by each cooperative and are only unique within it, so every event carries the cooperative too: `farmer.linked` sends `coop_code` + `farmer_id`, and later events send the `farmer_ref` we return (`KK-ID001`).

Languages: `om` (Afaan Oromoo), `am` (Amharic), `en` (English).

Events

`farmer.linked` - implemented

The farmer was validated against their cooperative's list (see Linking a farmer). We check the ID against the list, link it to the Telegram account and register the farmer on chain. Sent once per farmer.

`data`:

Field	Type	Notes
`coop_code`	string	Cooperative code: `KK` (Kilole Kirkir), `DM` (Dandi Misoma) or `KN` (Kananisa). Case-insensitive.
`farmer_id`	string	The cooperative ID, as typed, chosen or dictated. Spaces and case are normalized (`" id001 "` → `ID001`).
`telegram_user_id`	string of digits	Telegram's numeric user ID, as a string (it can exceed JS's safe integer range).
`telegram_username`	string, optional	
`language`	string, optional	Language the farmer chose: `om`, `am` or `en`.

`result`:

```
{
  "farmer_ref": "KK-ID001",
  "coop_code": "KK",
  "tx_hash": "c511e6a2...683c",
  "explorer_url": "https://stellar.expert/explorer/testnet/tx/c511e6a2...683c",
  "already_linked": false
}
```

Store `farmer_ref` for the Telegram account. Sending `farmer.linked` again for an already linked farmer and the same Telegram account returns the same `tx_hash` with `already_linked: true`, so it is safe to repeat.

Error code	HTTP	Meaning for the bot
`COOP_NOT_FOUND`	404	Unknown cooperative code.
`FARMER_NOT_FOUND`	404	The ID is not on that cooperative's list. The cooperative member helping with onboarding should check the list; do not let the farmer keep guessing IDs.
`FARMER_REF_TAKEN`	409	That ID is already linked to another Telegram account: the cooperative has to resolve it.
`TELEGRAM_ALREADY_LINKED`	409	This Telegram account is already linked to another farmer: the cooperative has to resolve it.
`INVALID_PAYLOAD`	400	A field is missing or invalid (see `details`).

`farmer.consent` - week 2

CofiBlocks provides the consent texts and their versions in the three languages. The bot presents them, records the farmer's explicit answer and when it was given, and sends it to us. We store it and record its hash on chain.

```
{
  "farmer_ref": "KK-ID001",
  "consent_version": "2026-11-v1",
  "language": "om",
  "accepted": true,
  "answered_at": "2026-11-24T08:15:00Z",
  "declarations": {
    "data_processing": true,
    "no_deforestation_since_2020_12_31": true,
    "information_is_accurate": true
  },
  "transcript": "optional text of what the farmer said",
  "audio_ref": "optional VoiceLedger reference to the recording"
}
```

- `consent_version` is the version of the text the bot showed; `answered_at` is when the farmer answered (ISO 8601).
- `accepted` must be an explicit answer from the farmer (a button or a clear spoken yes/no), never a default.
- The declarations are provisional until we send the final consent texts.

`farmer.profile` - week 2, limited questionnaire

Provisional questions. The existing data is described above. CofiBlocks will confirm the practice questions and whether any coffee-plot information actually needs collecting before VoiceLedger builds this part of the conversation.

This event carries the additional farm answers or corrections, not a repeat of the participant registration form. Do not ask for household size, the imported land area or a new GPS point by default. The example below illustrates an update where coffee-plot information really is missing; it does not mean every farmer must answer all these questions.

```
{
  "farmer_ref": "KK-ID001",
  "plots": [
    {
      "name": "Plot 1",
      "size": { "value": 1.5, "unit": "fachasa" },
      "location": { "lat": 7.86, "lng": 36.62, "source": "shared" }
    }
  ],
  "practices": { "shade_trees": true, "fertilizer": "compost" },
  "transcript": "optional",
  "audio_ref": "optional"
}
```

- Each update uses a new `event_id`. Omitted top-level fields keep their existing values. If `plots` is supplied, it is the complete updated list of plots; if `practices` is supplied, it is the complete updated set of agreed practice answers. Do not send empty placeholders for information that was not collected.
- For a newly collected or corrected point, `location.source` is `shared` (Telegram "Share location"), `typed` or `dictated`. Ask for the coffee plot's location, not the farmer's current location unless they are at the plot. CofiBlocks checks that the point falls inside Gomma woreda.
- Send a newly collected area with its original unit, such as `fachasa` in this example. Confirm the local unit name and conversion factor with the cooperative before testing this question. CofiBlocks handles any conversion needed internally and preserves the original value and unit; the farmer is not asked to convert to hectares. Imported areas must likewise not be converted until their actual source units are confirmed.
- The `shade_trees` and `fertilizer` fields illustrate practice answers; the final questions and answer choices will be supplied by CofiBlocks.

`delivery.created` - week 2

The farmer delivered coffee at a collection center. The bot collects the center and the estimated quantity, reads them back, and sends the event only after the farmer confirms. We generate the batch number and record the delivery on chain. The final weight is recorded later by the cooperative staff in the CofiBlocks web app, and the farmer is told about it (see Notifications).

```
{
  "farmer_ref": "KK-ID001",
  "collection_center": "Kirkir",
  "estimated_kg": 45,
  "ripeness": "ripe",
  "transcript": "optional",
  "audio_ref": "optional"
}
```

- `collection_center` must be one of the farmer's cooperative's centers in the reference table.
- `estimated_kg` is the farmer's estimate; it is not the recorded weight.
- If the answer is pending or a 5xx, follow the retry rules with the same `event_id` to avoid duplicate recording. Each separate delivery is a new event, including a second delivery on the same day.

`result` will include a unique `batch_id` for the delivery, `tx_hash` and `explorer_url`. CofiBlocks generates the batch ID, including how multiple deliveries on the same day are distinguished. VoiceLedger stores and displays the returned value without constructing it or relying on its format.

Notifications back to VoiceLedger

Needed for the pilot: when the cooperative staff record the weight of a delivery, the bot tells the farmer, for example "Your delivery 01-12-2026 KK-ID001 was received at Kirkir: 45 kg".

Optional for the pilot: updates for the later stages of a batch (washing, drying, storage, sale).

The mechanism is to be agreed: either we call a VoiceLedger webhook signed with the same HMAC scheme, or VoiceLedger polls an endpoint with each farmer's deliveries. Tell us which suits you.

What we need back from VoiceLedger

1. **Languages:** which languages the bot supports today, for voice recognition and for text.
2. **The technical contract:** acceptance of this document, or the changes you need.
3. **The linking flow:** your proposal for the cooperative-assisted onboarding (see Linking a farmer).
4. **Notifications:** webhook or polling.
5. **Transcripts and audio:** whether you will send the transcript and/or a reference to the audio. We store them off chain as backup evidence.
6. **A date for a joint test** of the bot against our staging.

Joint test

The minimum test we would run together, with a real Telegram account and a farmer from a test list:

1. Link a participant through the assisted onboarding.
2. Record their consent.
3. Run the agreed short farm questionnaire, verify that imported household/land/location data is not asked for again, and submit an additional answer or correction.
4. Register a delivery, retry it with the same `event_id`, and check that it is recorded only once.
5. Record the weight in the CofiBlocks web app and check that the farmer gets the confirmation.

Testing it locally (CofiBlocks)

With our backend running, we sign and send events exactly as the bot would:

```
npm run voiceledger:send -- farmer.linked
'{"coop_code":"KK","farmer_id":"ID001","telegram_user_id":"123456789"}'
npm run voiceledger:send -- --status <event_id>
```